

به نام خدا

گزارش تمرین کامپیوتری چهارم

سیگنال‌ها و سیستم‌ها-دکتر اخوان

امیرمحمد میرحسینی ۸۱۰۱۰۲۶۰۱

تمرین ۱-۱

مانند تمرین های گذشته مپست را ایجاد می کنیم: تصاویر کد و خروجی مپست ضمیمه شده اند.

```
data = cell(2,32);

row1 = 'abcdefghijklmnopqrstuvwxyz .,:!";
for i = 1:length(row1)
    data{1, i} = row1(i);
end

for i = 0:31
    data{2, i+1} = dec2bin(i, 5);
end

save('Mapset.mat', 'data');
```

Columns 1 through 11											
{ 'a' }	{ 'b' }	{ 'c' }	{ 'd' }	{ 'e' }	{ 'f' }	{ 'g' }	{ 'h' }	{ 'i' }	{ 'j' }	{ 'k' }	
{ '00000' }	{ '00001' }	{ '00010' }	{ '00011' }	{ '00100' }	{ '00101' }	{ '00110' }	{ '00111' }	{ '01000' }	{ '01001' }	{ '01010' }	
Columns 12 through 22											
{ 'l' }	{ 'm' }	{ 'n' }	{ 'o' }	{ 'p' }	{ 'q' }	{ 'r' }	{ 's' }	{ 't' }	{ 'u' }	{ 'v' }	
{ '01011' }	{ '01100' }	{ '01101' }	{ '01110' }	{ '01111' }	{ '10000' }	{ '10001' }	{ '10010' }	{ '10011' }	{ '10100' }	{ '10101' }	
Columns 23 through 32											
{ 'w' }	{ 'x' }	{ 'y' }	{ 'z' }	{ ' ' }	{ '.' }	{ ',' }	{ '!' }	{ ':' }	{ '"' }		
{ '10110' }	{ '10111' }	{ '11000' }	{ '11001' }	{ '11010' }	{ '11011' }	{ '11100' }	{ '11101' }	{ '11110' }	{ '11111' }		

تمرین ۲-۱

```
function coded = coding_amp(msg, speed)
    load("Mapset.mat");

    len = strlen(msg);
    bitcount = 5 * len;
    binmsg = zeros(1, bitcount);

    k = 1;
    for i = 1:len
        ch = extractBetween(msg, i, i);
        idx = find(strcmp(data(1, :), ch));

        binBits = data{2, idx};
        for p = 1:5
            binmsg(k) = binBits(p);
            k = k + 1;
        end
    end
    binmsg = binmsg-48;
    % coded = binmsg;

    block_count = bitcount / speed;
    ts = zeros(1, 100 * block_count);

    for i = 1:block_count
        bitblock = binmsg((i-1)*speed + 1 : i*speed);
        str = num2str(bitblock);
        str(str == ' ') = [];
        %disp(bitblock);

        amp_level = bin2dec(str);

        amplitude = amp_level / (2^speed - 1);

        for j = 1:100
            ts((i-1)*100 + j) = amplitude * sin(2 * pi * (j-1) / 100);
        end
    end

    figure;
    plot(ts);
    title(['Signal Plot (Speed = ', num2str(speed), ')']);
    xlabel('Sample Index');
    ylabel('Amplitude');

    coded = ts;
end
```

ورودی تابع پیام و سرعت انتقال آن بر حسب بیت بر ثانیه می باشد. ابتدا لود می کنیم مپست بخش قبل را. سپس چون هر حرف پیام در باینری به ۵ بیت تبدیل می شود یک متغیر به نام *bitcount* تعریف می کنیم. برای باینری کردن پیام یک متغیر به نام *binmsg* ایجاد می کنیم. سپس در حلقه به کمک دستور *extractBetween* هر حرف پیام را مشخص کرده و در مپست (سطر ۱ که حروف قرار دارند) به دنبال ایندکس آن می گردیم. سپس به کمک ایندکس پیدا

شده سطر ۲ مپ ست با همان ایندکس باینری شده حرف مربوطه را بدست می آوریم و پیام باینری شده را در *binmsg* ذخیره می کنیم.

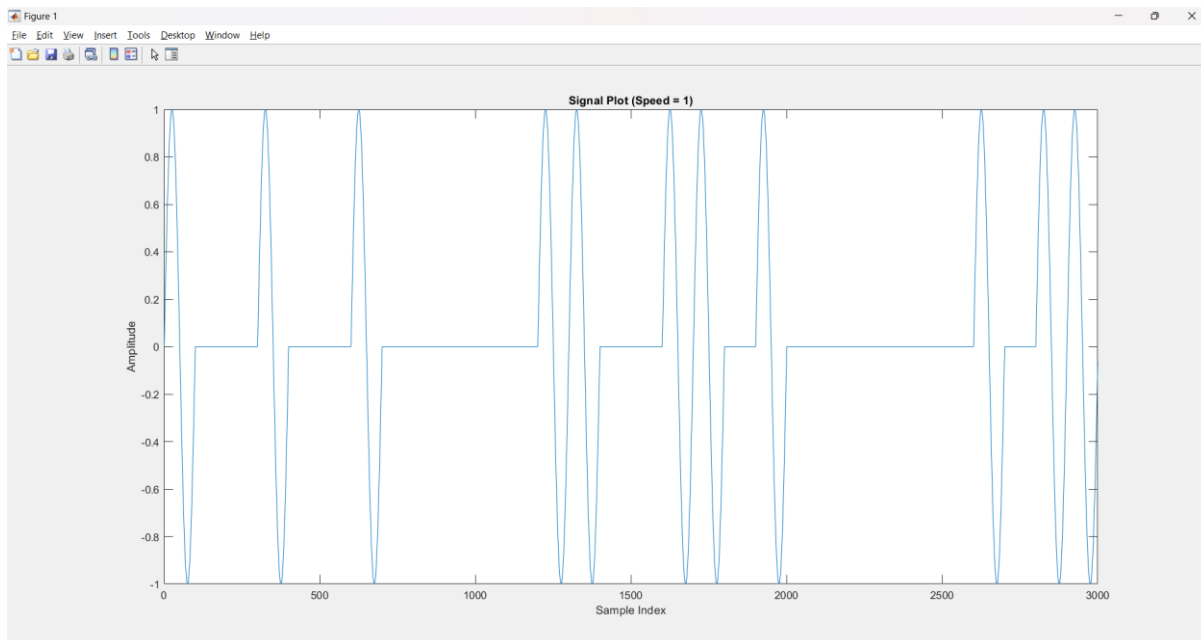
همانطور که می دانیم با توجه به سرعت، تعداد بیت های پیام می تواند متفاوت باشد. یعنی کلمه *signal* به طور پیشفرض ۳۰ بیت باینری نیاز دارد. اما با توجه به سرعت می توانیم در زمان کمتری پیام را ارسال کنیم. متغیر *block_count* به همین سبب ایجاد شده است که از تعداد بیت پیام تقسیم بر سرعت محاسبه می گردد.

با توجه به اینکه فرکانس برابر ۱۰۰ هرتز می باشد. لذا ثانیه های مورد نیاز برای نمایش سیگنال کدگذاری شده را با آرایه *ts* به طول ۱۰۰ ضربدر *block_count* ایجاد می کنیم. در بخش بعد در حلقه به اندازه *speed* بیت پیام باینری شده را جدا جدا می کنیم. هر بخش را به دسیمال تبدیل می کنیم و تقسیم بر 2^{speed} می کنیم و *amplitude* را محاسبه می کنیم. سپس به *ts* برای ثانیه مربوطه به اندازه $amplitude * \sin(200 * \pi * t)$ مقدار می دهیم. (مقدار دهی در کد در داخل ضابطه سینوس کمی متفاوت است ولی به طور کلی همین است).

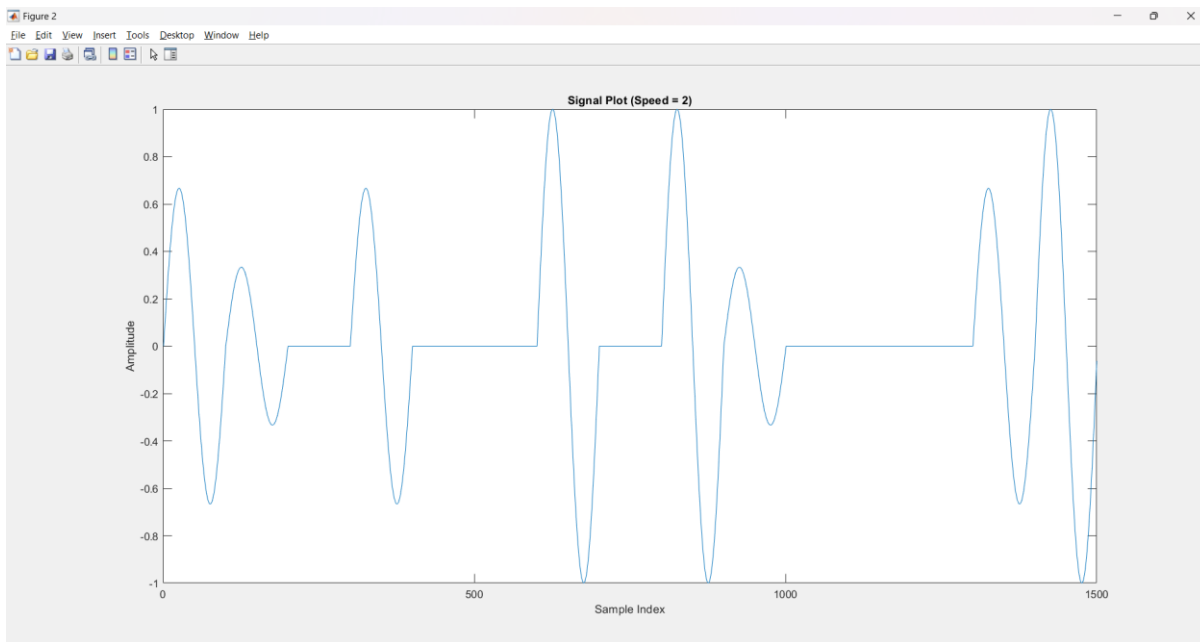
سپس سیگنال کدگذاری به دست می آید و آن را در متغیر خروجی به صورت *coded=ts* ذخیره می کنیم. همچنین پس از اجرای تابع نمودار سیگنال کدگذاری شده نمایش داده می شود.

تمرین ۱-۳)

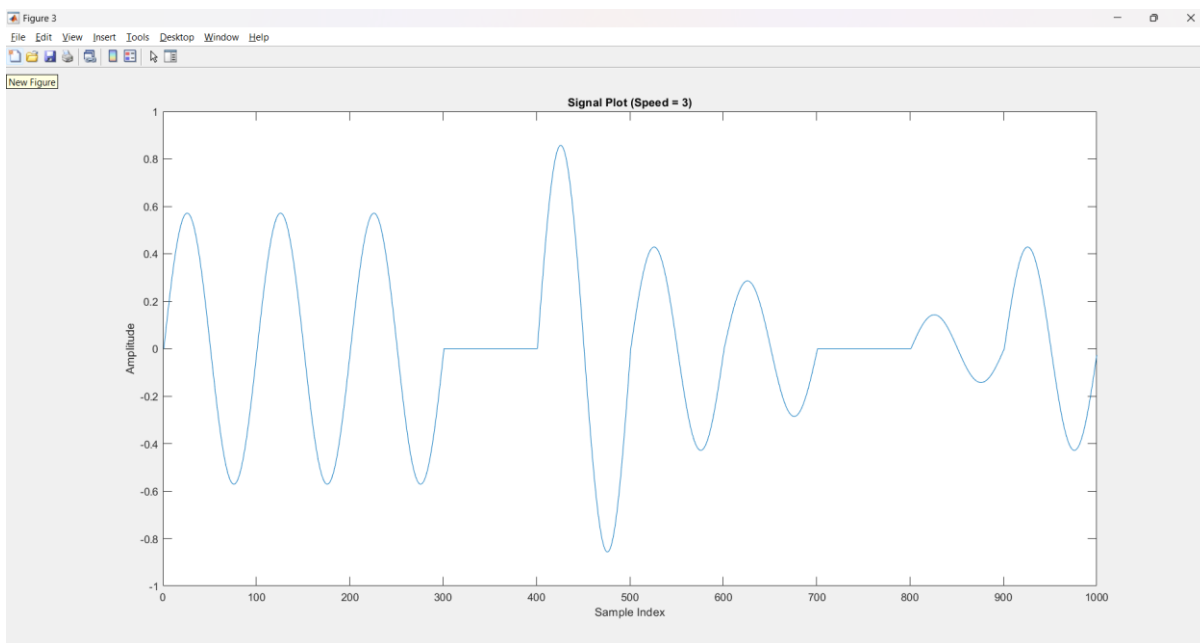
سرعت ۱ بیت بر ثانیه



سرعت ۲ بیت بر ثانیه



سرعت ۳ بیت بر ثانیه



همانطور که مشاهده می شود با افزایش سرعت تنوع دامنه ها بیشتر می شود و زمان ارسال سیگنال نیز کمتر می شود.

```
---,
msg = 'signal';
sgn1 = coding_amp(msg, 1);
sgn2 = coding_amp(msg, 2);
sgn3 = coding_amp(msg, 3);
```

تمرین ۱-۴

```
function decoded = decoding_amp(signal , speed)
    len = length(signal);
    blockcount = len/100;
    cor_sample = zeros(1, 100);
    for i = 1:100
        cor_sample(i) = 2*sin(2*i/100*pi);
    end

    bitcount = blockcount*speed;
    corlist = zeros(1,blockcount);
    for i = 1:blockcount
        subt = signal((i-1)*100+1:100*i);
        cor = abs(subt*(cor_sample'));
        corlist(i)=cor;
    end

    % disp(corlist);
    if speed == 1
        cor_value = round((corlist/100));
    else
        cor_value = round((corlist/100)*(2^speed-1));
    end
    binmsg=[];
    % disp(cor_value);
    for i = 1:blockcount
        bin_part = dec2bin(cor_value(i),speed);
        binmsg = [binmsg bin_part];
    end

    % decoded=binmsg;
    finalmsg = [];
    % decoded=binmsg;
    finalmsg = [];
    load("Mapset.mat");
    for i = 1:(bitcount/5)
        subl = binmsg(5*(i-1)+1:5*(i-1)+5);

        jdx = find(strcmp(data(2, :), subl));
        letter = data(1, jdx);
        finalmsg = [finalmsg letter];
    end
    decoded = finalmsg;
    % disp(decoded);
end
```

تابع *decoded_amp* : ورودی ها سیگنال و سرعت ارسال آن می باشد.

ابتدا طول سیگنال را در متغیر *len* ذخیره می کنیم. سپس چون فرکانس ۱۰۰ هرتز است، با $len/100$ ، *blockcount* (هرثانیه سیگنال) محاسبه می کنیم. در قسمت بعد سمپل مورد نیاز برای کورلیشن گرفتن را ایجاد می کنیم که تابع $2\sin(200\pi t)$ برای ۱ دوره تناوب (۱۰ تا ۱۰۰۰ میلی ثانیه) می باشد.

می دانیم که در یک ثانیه به اندازه *speed* بیت پیام منتقل می شود. بنابراین $bitcount=blockcount*speed$.

سپس یک متغیر به نام *corlist* برای محاسبه کورلیشن هر ثانیه از سیگنال طبق راهنمای صورت سوالات ایجاد می کنیم.

وارد حلقه می شویم. به ازای هر ثانیه (یا ۱۰ تا ۱۰۰۰ میلی ثانیه) کورلیشن سیگنال ثانیه مربوطه (در متغیر *subt* سیگنال هر ثانیه را برای کورلیشن جدا کردیم) را سیگنال سمپل به دست می آوریم و در *corlist* به صورت یک المنت از آرایه ذخیره می کنیم.

همانطور که می دانیم عدد کورلیشن در مقیاس مورد نظر ما نمی باشد لذا با تقسیم بر ۱۰۰ به دامنه سیگنال هر ثانیه می رسیم. دامنه سیگنال ثانیه مربوطه را در 2^{speed} ضرب می کنیم تا از فرم کسری خارج و به عدد دسیمال صحیح (با کمک دستور *round*) می رسیم. عدد دسیمال را به عدد باینری تبدیل کرده و پیام کدگذاری ثانیه مربوطه را به صورت باینری به دست می آوریم. در بخش بعد اعداد *speed* بیتی باینری به دست آمده را در کنار هم به ترتیب قرار داده و در متغیر *binmsg* ذخیره می شود. حال باید پیام باینری را به پیام اصلی (حروف الفبای انگلیسی) تبدیل کنیم.

برای این منظور در حلقه عدد باینری به دست آمده را به صورت ۵ بیت ۵ بیت جدا کرده و هر ۵ بیت را با *strcmp* با سطر دوم مپ ست که باینری شده حروف الفبا و نمادها بودند مقایسه می کنیم و ایندکس مربوطه را به دست می آوریم. سپس در سطر اول مپ ست و ایندکس به دست آمده ، حرف الفبای مربوط به آن ۵ بیت را به دست می آوریم. و در انتها

حروف را کنار هم چیده و در *finalmsg* ذخیره می‌کنیم. خروجی تابع که متغیر *decoded* می‌باشد، پیام رمزگشا شده سیگنال می‌باشد.

تست فانکشن:

7	ss = decoding_amp(sgn2 , 2);	
8	ss1 = decoding_amp(sgn3 , 3);	
9	s2 = decoding_amp(sgn1 , 1);	
10	fprintf('decoded message for 3 bit/sec variable is: %s\n: ' , ss1);	
11		
12		
13		
14	%%	
15	ex = zeros(1,3000);	
16	ex = ex + randn(1,3000);	
17	showplot(ex);	
18	disp(mean(ex));	
19	disp(var(ex));	
20		
21	%%	
22	noised_sgn1 = sgn1 + 0.0001*(randn(1,length(sgn1)));	
23	noised_sgn2 = sgn2 + 0.0001*(randn(1,length(sgn2)));	
24	noised_sgn3 = sgn3 + 0.0001*(randn(1,length(sgn3)));	
25	noisedec1 = decoding_amp(noised_sgn1 , 1);	
26	noisedec2 = decoding_amp(noised_sgn2 , 2);	
27	noisedec3 = decoding_amp(noised_sgn3 , 3);	
28	%%	
29	flag1 = 0;	
30	flag2 = 0;	
31	flag3 = 0;	
32	for i = 2000:200:15000	
33		
34	temp1noise = sgn1 + 0.0001*i*(randn(1,length(sgn1)));	
mmand Window		
decoded message for 3 bit/sec variable is: signal		

به درستی کار می‌کند.

تمرین ۱-۵)

```
function showplot(signal)
    figure;
    subplot(1,2,1);
    plot(signal);
    title(['Signal Plot']);
    xlabel('Sample Index (time)');
    ylabel('Amplitude');

    subplot(1,2,2);
    histogram(signal, 50);
    title('Histogram of signal');
    xlabel('Value');
    ylabel('Frequency');

end
```

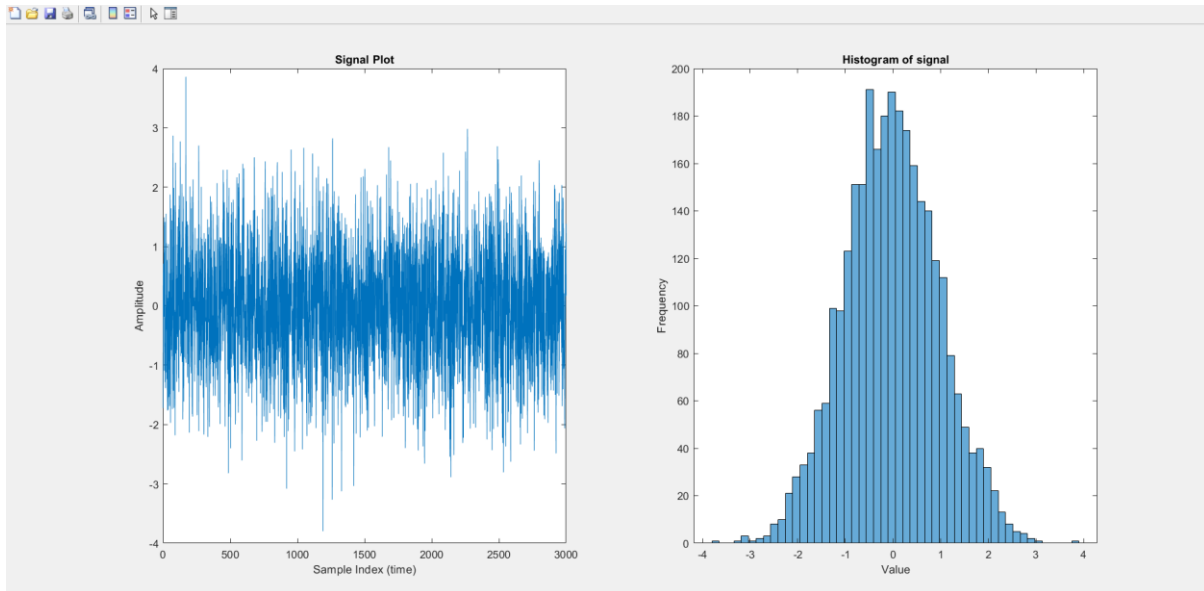
-0.0291
0.9752

```
%%
ex = zeros(1,3000); |
ex = ex + randn(1,3000);
showplot(ex);
disp(mean(ex));
disp(var(ex));
```

(الف) هیستوگرام (نمودار سمت راست) گاوسی است.

(ب) میانگین صفر است. (قله توزیع نرمال روی میانگین قرار می گیرد.) ، همچنین میانگین که در کد محاسبه شده نیز (عکس بالا) تقریباً برابر صفر است. (-0.0291)

(ج) واریانس محاسبه شده در کد تقریباً برابر ۱ است. (0.9752)



تمرین ۱-۶)

```
%%
noised_sgn1 = sgn1 + 0.01*(randn(1,length(sgn1)));
noised_sgn2 = sgn2 + 0.01*(randn(1,length(sgn2)));
noised_sgn3 = sgn3 + 0.01*(randn(1,length(sgn3)));
noisedec1 = decoding_amp(noised_sgn1 , 1);
noisedec2 = decoding_amp(noised_sgn2 , 2);
noisedec3 = decoding_amp(noised_sgn3 , 3);
```

noisedec1	'signal'	12 1x6	d
noisedec2	'signal'	12 1x6	d
noisedec3	'signal'	12 1x6	d

تمرین ۱-۷) و تمرین ۱-۸) همانطور که مشاهده می‌شود هرچه سرعت بیشتر باشد، مقاومت سیگنال در برابر نویز کمتر است و با افزایش نویز رمزگشایی سیگنال با اختلال بیشتری مواجه می‌شود. که با آنچه در مقدمه صورت سوالات بیان شد همخوانی دارد.

به طوری که با انحراف معیار

۰.۲ تا ۰.۴ سیگنال با بیت ریت ۳ در شناسایی دچار مشکل می‌شود.

۰.۴ تا ۰.۶ سیگنال با بیت ریت ۲ در شناسایی دچار مشکل می‌شود.

۱.۱ تا ۱.۴ سیگنال با بیت ریت ۱ در شناسایی دچار مشکل می‌شود.

(مقدار i برابر انحراف معیار بر حسب صدم می‌باشد.)

```

decoded message for 1 bit/sec with i= 22 is: signal
: decoded message for 2 bit/sec with i= 22 is: signal
decoded message for 3 bit/sec with i= 22 is: wignal
decoded message for 1 bit/sec with i= 54 is: signal
: decoded message for 2 bit/sec with i= 54 is: sienal
decoded message for 3 bit/sec with i= 54 is: oifrae
decoded message for 1 bit/sec with i= 132 is: sygnal
: decoded message for 2 bit/sec with i= 132 is: kieovf
decoded message for 3 bit/sec with i= 132 is: veuoe.

```

```

%%
flag1 = 0;
flag2 = 0;
flag3 = 0;
for i = 20:2:150

    temp1noise = sgn1 + 0.01*i*(randn(1,length(sgn1)));
    temp2noise = sgn2 + 0.01*i*(randn(1,length(sgn2)));
    temp3noise = sgn3 + 0.01*i*(randn(1,length(sgn3)));
    dc1 = decoding_amp(temp1noise , 1);
    dc2 = decoding_amp(temp2noise , 2);
    dc3 = decoding_amp(temp3noise , 3);

    if flag3==0 && dc3 ~= "signal"
        fprintf('decoded message for 1 bit/sec with i= %d is: %s\n' , i,dc1);
        fprintf('decoded message for 2 bit/sec with i= %d is: %s\n' , i,dc2);
        fprintf('decoded message for 3 bit/sec with i= %d is: %s\n' , i,dc3);
        flag3=1;
    end
    if dc2 ~= "signal" && flag2==0
        fprintf('decoded message for 1 bit/sec with i= %d is: %s\n' , i,dc1);
        fprintf('decoded message for 2 bit/sec with i= %d is: %s\n' , i,dc2);
        fprintf('decoded message for 3 bit/sec with i= %d is: %s\n' , i,dc3);
        flag2=1;
    end
    if dc1 ~= "signal" && flag1==0
        fprintf('decoded message for 1 bit/sec with i= %d is: %s\n' , i,dc1);
        fprintf('decoded message for 2 bit/sec with i= %d is: %s\n' , i,dc2);
        fprintf('decoded message for 3 bit/sec with i= %d is: %s\n' , i,dc3);
        flag1=1;
    end
end
end

```


تمرین ۱-۹) اگر فرستنده قدرت بیشتری برای ارسال پیام داشت، مشکل برطرف می‌شد. زیرا تاثیر نویز روی دامنه بیشتر کمتر می‌باشد و دقت رمزگشایی سیگنال‌ها بیشتر می‌شود. (با افزایش دامنه یا همان قدرت)

تمرین ۱-۱۰) تا مقدار دلخواه می‌توان افزایش داد. اما نکته این است که نیازی به انجام اینکار برای اعداد بسیار بالا نیست چون مثلا نیاز به رمزگشایی عدد چند هزار بیتی خواهیم بود و پردازش برنامه سنگین تر می‌شود.

تمرین ۱-۱۱) نه، این کار فایده‌ای ندارد، زیرا قدرت سیگنال رو بیشتر نمی‌کند. فقط عددی که از *correlation* درمیاد فرق می‌کنن و بازه‌شون عوض می‌شه. یعنی ممکنه مقدار *correlation* بیشتر بشه، ولی چون رو دقت یا اون *threshold* که تعیین کردیم اثری نمی‌ذاره، نتیجه تغییری نمی‌کنه.

در عمل نیز برای مثال صورت سوال آزمایش کردیم و نتیجه‌ای در انحراف معیار تغییر سیگنال‌ها نداشت. بنابراین بی تاثیر است.

تمرین ۱-۱۲) معمولا در بازه‌ی ۱ تا ۱۶ مگابیت بر ثانیه (*Mbps*) است. ما در سیگنال‌های کدگذاری شده در این تمرین حداکثر با ۳ بیت بر ثانیه پیام را به صورت رمزگذاری شده منتقل کردیم. (هرچند کد ایجاد شده توانایی ارسال هر سرعتی را دارد اما در پردازش‌های سنگین‌تر یعنی با سرعت بالاتر احتمال ایجاد مشکل وجود دارد).